uni**Log**

Submission answers

Every answer below is written from the working codebase, not from intent. Figures marked *verified* come from real runs; anything unmeasured is labelled as such, so nothing on a slide can be challenged in Q&A.

01 — BRIEF ABOUT YOUR SOLUTION

uniLog is an automated product-data enrichment engine for industrial catalogs.

It takes a supplier's raw export — six columns of inconsistent identity data — and produces the client's 252-column commerce-ready delivery format, with no human touching a row that the system is confident about.

Seven stages run in sequence: **sanitize, classify, retrieve, extract, normalize, validate, export**. The design principle running through all of them is a split of responsibility: **AI proposes, deterministic code disposes.** A language model is used at exactly one point — reading a technical document and proposing candidate facts. It never writes to the output. Every value it proposes then passes through rule-based normalization, conflict detection and a quality gate before any column is filled.

The result is a catalog where each published value can be traced back to the document, page and text snippet it came from, and where the rows a machine should not have decided alone are routed to a person with the reason attached.

INPUT

6 columns

Part number, description, three brand fields that are usually placeholders, and a manufacturer name carrying an ERP code.

OUTPUT

252 columns

The client's delivery format, read from their own sheet at runtime so the deliverable cannot drift from the contract.

HUMAN EFFORT

Only where earned

Green publishes automatically. Amber queues for review with a stated reason. Red is flagged for research.

---

02 — HOW IT ENRICHES MINIMAL PRODUCT INFORMATION

Each stage adds one kind of knowledge the previous stage could not have had.

The clearest way to answer this is to follow one real row from the sample dataset all the way through. This is an actual product from the input file and an actual output row from a verified run.

The input

```
Mfg_Part_Num  DCB518ASTS06G
Part_Desc     DCB518ASTS06G Diablo 1/2"x18" - Sanding Belt 6pc
E1_Brand      -- Unbranded --
Unilog_Brand  -- No Unilog Brand --
DIB_Brand     -- No DIB Brand --
Part_Manuf    Freud Inc (2435)
```

Three of the six fields carry no information at all. The manufacturer field carries an internal ERP code. Everything a retailer would actually want — grit, dimensions, backing material, pack quantity — is either buried in one string or absent entirely.

What each stage contributes

Stage	Knowledge added	Result on this row
1 — Sanitize	Removes noise that would poison every later stage	Placeholders dropped; ERP suffix stripped → `Freud Inc`
2 — Classify	Decides *which of the 252 columns even apply* to this product	Category `Abrasives`; target attributes become Grit, Diameter, Arbor Size, Abrasive Material, Max RPM; classpath `Abrasives>Coated Abrasives>Discs & Belts`
3 — Retrieve	Brings in authoritative text that does not exist in the input at all	Found and downloaded the manufacturer's own page, `diablotools.com/products/DCB518ASTS06G`; marketplace results rejected
4 — Extract	Turns prose into typed facts with provenance	Width 1/2 IN, Length 18 IN, Quantity 6 EA, Product Type, Brand — each with confidence, source snippet and page
5 — Normalize	Makes values comparable across suppliers	`inches → IN`, `6pc → 6 EA`, fractions preserved as `1/2` per the delivery convention
6 — Validate	Decides whether this row is fit to publish	Scored on completeness, confidence and normalization → routed green
7 — Map & export	Places every fact in the column the contract expects	Facts land in the ATTRIBUTE_LABEL / VALUE / UOM triplets; dimensions, classpath and five description fields filled

The output

```
Mfg_Part_Num       DCB518ASTS06G
BRAND_NAME         Diablo            ← recovered from the description, not the distributor
MANUFACTURER_NAME  Freud Inc
Classpath          Abrasives>Coated Abrasives>Discs & Belts
Product Name       Sanding Belt
INVOICE_DESC       SANDING BELT 1/2IN 18IN 6EA
SHORT_DESC         Diablo Sanding Belt DCB518ASTS06G 1/2 IN, 18 IN, 6 EA
ATTRIBUTE 1        Width  / 1/2 / IN
ATTRIBUTE 2        Length / 18  / IN
LENGTH / WIDTH     18 IN / 1/2 IN
MFR URL            https://diablotools.com/products/DCB518ASTS06G
```

One detail worth calling out on the slide: **the brand.** All three supplier brand columns were placeholders, so a naive fallback lands on Freud Inc — the distributor. uniLog cross-checks the brand parsed out of the product text against the resolved one and publishes Diablo, which is

what a customer actually searches for. That is enrichment producing a value that was in no structured field anywhere in the input.

---

03 — ACCURACY AND TRUST

Six independent layers, arranged so that no single failure reaches the catalog.

The validation strategy is deliberately layered rather than relying on any one mechanism — particularly not on the language model's own self-assessment.

1 — Source control (before any AI runs)

The engine will not read from marketplaces. A deny-list of twenty domains — Amazon, eBay, Grainger, McMaster, Home Depot, Zoro, Fastenal and others — is applied at both discovery and download time, and retrieved results are ranked to prefer the manufacturer's own domain and PDF documents. Industrial specs on resale listings are frequently wrong or transcribed; refusing them at the source removes an entire class of error.

2 — Rule-based validation (deterministic, no AI)

Stage 1 and Stage 5 are pure Python with no model in the loop. Placeholder detection, ERP-code stripping, ~55 unit conversions, a ~30-term material vocabulary, imperial fraction handling and character-limit enforcement are all provable transformations. Anything the vocabulary does not recognise is not silently accepted — it is passed through and flagged UNMAPPED_ENUM, which lowers the product's score and surfaces the vocabulary gap.

3 — Confidence scoring at the fact level

Every extracted fact carries a High / Medium / Low confidence, the exact source snippet it came from, the page number, and which document produced it (retrieved_document vs supplier_description). Confidence is scored per fact, not per product, so one weak value cannot hide behind six strong ones.

4 — Multi-source verification and conflict detection

When more than one document is retrieved for a product, facts from each are extracted independently. If the same attribute comes back with two different normalized values, that attribute is recorded as a conflict, the product is penalised ten points per conflicting attribute, and the conflicting attribute names are named in the report. Agreement across independent sources raises trust; disagreement is surfaced rather than silently resolved by picking one.

5 — Composite quality score

```
score = completeness_ratio  × 40      how many required attributes were found
      + confidence_ratio    × 40     share of facts at High/Medium
      + normalization_ratio × 20      share that mapped to known vocabulary
      - 10 per conflicting attribute
      clamped to 0–100
```

6 — Human review routing (traffic light)

● SCORE ≥ 80

AUTO_APPROVED

Publishes without human involvement. Specs are written into the delivery format.

● SCORE 50–79

HUMAN_REVIEW

Ships identity and taxonomy only, and queues with the specific reason: low completeness, low confidence, or the named conflict.

● SCORE < 50

FAILED

Fails closed. A product with zero extracted facts scores zero — it is never published on the assumption that no news is good news.

Underpinning all six: provenance and regression tests

Four intermediate files are written every run — retrieval_log.csv, extracted_facts.csv, normalized_facts.csv, validation_report.csv — plus a timestamped run log. When a retailer disputes a spec, the chain from published column back to source URL and snippet is already on disk. A 23-test suite covers the routing boundaries, the conflict penalty, fail-closed behaviour, unit and material mapping, and the export shape.

The model is never trusted to be right. It is only trusted to propose something that deterministic code can then check.

04 — HOW IT DIFFERS FROM EXISTING APPROACHES

Four approaches already exist. Each fails in a specific, nameable way.

Existing approach	Where it breaks	What uniLog does instead
Manual data-entry teams	Accurate but slow and expensive; quality drifts between operators and cannot be audited after the fact	Automates the confident majority and spends human attention only on rows the score flags
"Ask an LLM to fill the columns"	The model generates the row directly, so plausible-but-invented specs enter the catalog with no way to tell them from real ones	The model may only *extract from a retrieved document*. It never writes output. Deterministic code composes every published column
Classic PIM platforms (Akeneo, Salsify, inRiver)	Excellent at storing and governing attributes, but they assume the attributes already exist — they do not source them	Solves the acquisition problem that sits upstream of a PIM, and emits a file a PIM can ingest
Scraper / feed-aggregator tools	Take whatever the first result says, marketplaces included, and carry no confidence or provenance	Manufacturer-first sourcing with an explicit marketplace deny-list, plus per-fact confidence and a stored source URL

The three structural differences

- **The schema is read from the client's contract, not hardcoded.** The 252 column names are loaded from the delivery-format sheet at runtime. If the client revises the sheet, the output follows on the next run with no code change — and the deliverable can never silently drift from what was agreed.
- **The output is row-aligned with the input.** Every product that goes in comes out, whether or not it passed. Approved rows carry specs; the rest carry identity and taxonomy and sit in the review queue. Nothing disappears between input and delivery.
- **Confidence is a routing decision, not a display field.** Most tools show a confidence number and leave the judgement to a human scanning a spreadsheet. Here the score determines what publishes, so review effort is allocated by the system rather than by whoever is looking.

---

05 — HOW IT SOLVES THE PROBLEM STATEMENT

The brief asks for 6 columns to become 252 at

catalog scale, with quality that survives scrutiny.

Requirement	How uniLog meets it	Status
Transform 6 → 252 columns	Schema loaded from the client sheet; facts mapped into the 50 attribute triplets, dimension columns and five description fields	verified — 252 columns
Handle the full 1,000-product dataset	Stateless per-product processing; the whole file parses and classifies in seconds	stages 1–2, 5–7 verified at 1,000
Minimal human intervention	One command runs all seven stages; only amber and red rows reach a person	verified
Source real technical data	Autonomous retrieval finds and downloads manufacturer documentation, then extracts its text	verified on live products
Quality that can be defended	Six validation layers, a 0–100 score, and four provenance files per run	verified
Operate within free-tier API limits	Built-in 15 requests/minute limiter with server-hinted cooldown on quota errors	verified — 0 rate-limit failures

Economics for the slide. At the enforced free-tier rate of 15 requests per minute, a 1,000-product catalog completes extraction in roughly 67 minutes at zero API cost. The comparison point is a manual specialist working through the same catalog over weeks. Quote the time saving; treat any cost-reduction percentage as an estimate unless you have the client's own per-SKU labour figure to anchor it.

---

06 — USP

One line for the slide

AI proposes, deterministic rules dispose — and every published value can be traced back to the document, page and snippet it came from.

The three supporting claims

1. **Provenance by construction, not as a feature.** The audit trail is not a log bolted on afterwards; it is the data structure the pipeline passes between stages. A value cannot exist in the output without a recorded source.
2. **Manufacturer-first sourcing.** An explicit marketplace deny-list and authority ranking means the engine prefers the party legally responsible for the specification over the party reselling it.

3. **The contract drives the code.** The delivery schema is read from the client's own sheet each run, so the output shape is defined by the agreement rather than by a developer's copy of it.

07 — FEATURES

What the system does today

STAGE 1

Data sanitization

Placeholder detection across supplier fields, ERP-suffix stripping, and a brand fallback chain terminating in an explicit UNKNOWN.

STAGE 2

Taxonomy & classification

Seven categories, per-category target attributes, Dept / Class / Fine / Classpath assignment and product-name derivation.

STAGE 3

Autonomous document retrieval

Query construction with relaxed fallbacks, keyless web search, twenty-domain marketplace deny-list, manufacturer-domain and PDF ranking, politeness throttling, per-product document store.

STAGE 3

Multi-format text extraction

PDF via pypdf and HTML via BeautifulSoup, with navigation and script content stripped before the text reaches the model.

STAGE 4

Evidence-graph extraction

Structured facts with attribute, value, unit, confidence, source snippet, page number and originating document.

STAGE 4

Quota-aware API client

Requests-per-minute limiter, server-hinted cooldown on 429, exponential backoff with jitter on 5xx, and per-run statistics.

STAGE 5

Deterministic normalization

~55 unit conversions, ~30-term material vocabulary, imperial fraction preservation, grit-scale handling, word-boundary character limits.

STAGE 6

Conflict detection & scoring

Cross-source disagreement detection, weighted 0–100 quality score, per-conflict penalty, reasons attached to every flagged product.

STAGE 6

Traffic-light routing

Automatic three-way split into publish, review and research queues with a written justification per row.

STAGE 7

Contract-driven schema mapping

252 columns read from the client sheet, 50 attribute triplets filled by confidence rank, dedicated dimension columns, five generated description fields.

PLATFORM

Provenance & observability

Four intermediate CSVs per run, a per-product document store and a timestamped structured log file.

PLATFORM

Operator controls

Command-line flags for batch size, document retrieval, rate limit, offline plumbing runs and verbose output.

INTERFACE

Upload & validate UI

Browser front end that parses a supplier CSV client-side, checks the header against the input schema, previews rows and projects the 6 → 252 transformation.

QUALITY

Regression test suite

23 tests covering routing boundaries, conflict penalties, vocabulary mapping, export shape and rate-limiter behaviour.

Process flow

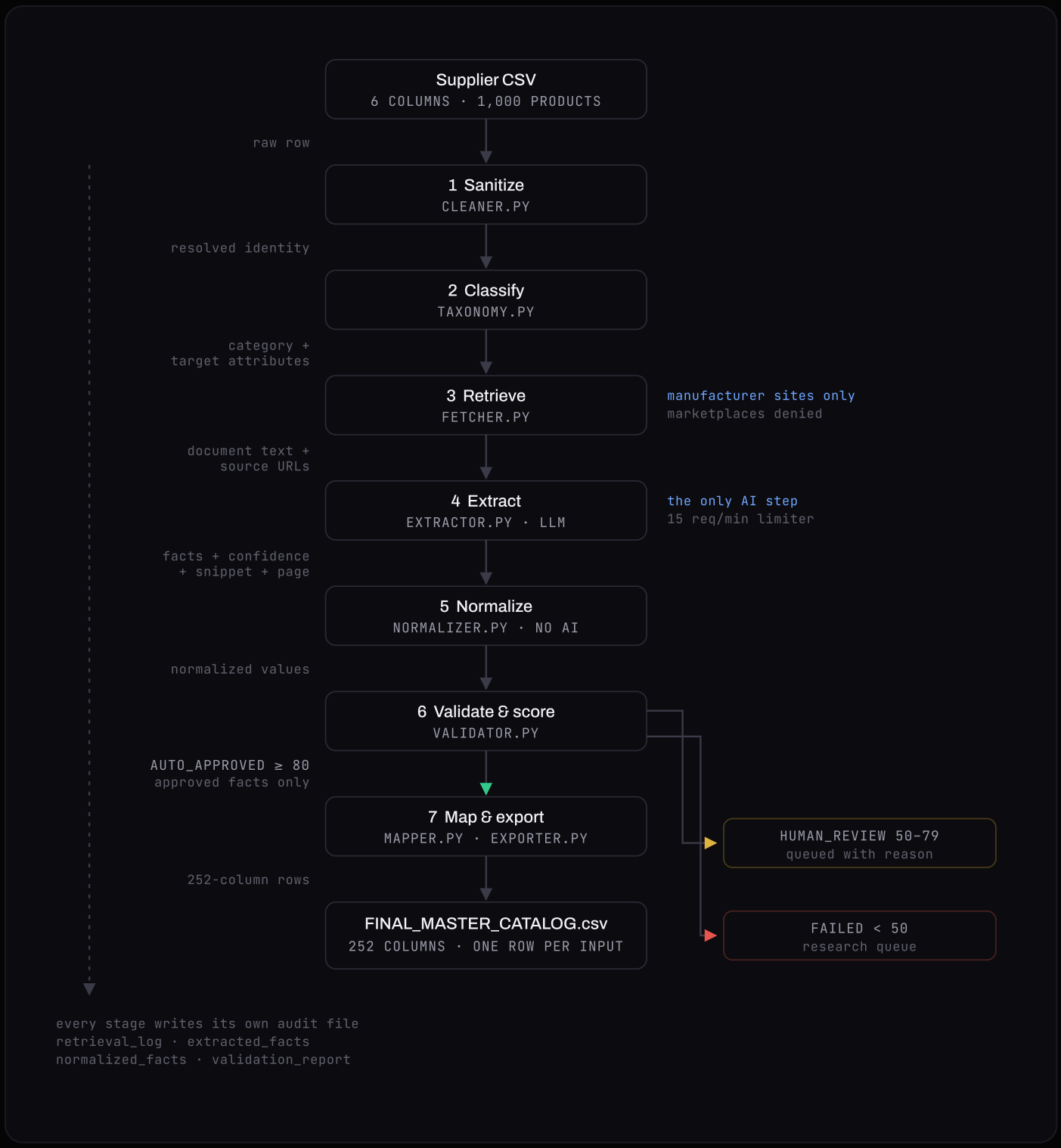

Figure 1. Each arrow is labelled with the artifact that actually moves between stages, not just sequence. Note the branch at Stage 6: only approved facts continue into the export, while amber and red rows leave the spine — and every stage writes to the provenance rail on the left, which is what makes any published value auditable afterwards.

Interface

The operator-facing surface is deliberately one screen. A catalog manager arrives with a supplier export and needs to know two things before committing to a run: whether the file is readable, and what it will become.

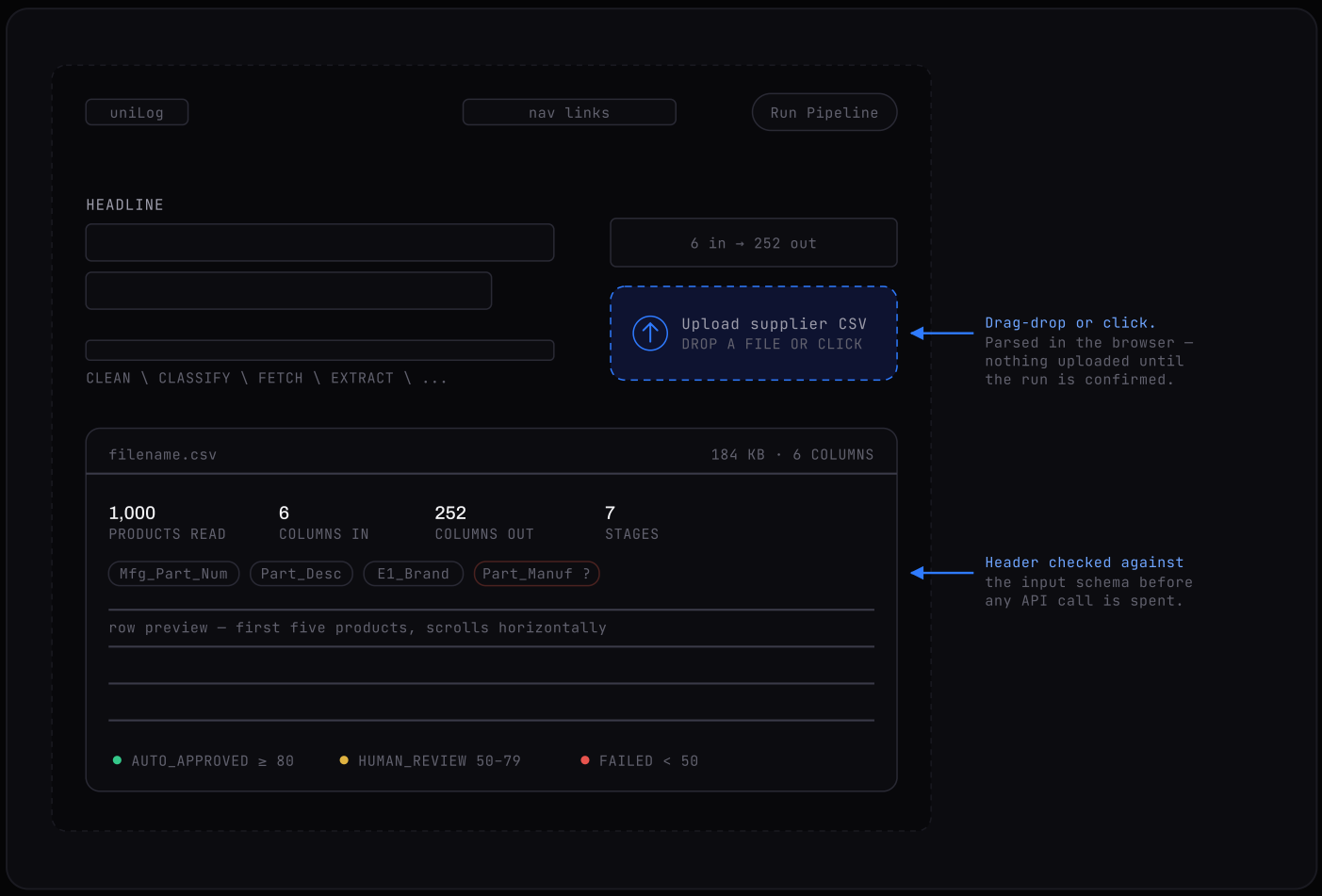

Figure 2. The upload screen. The file is parsed client-side, so a malformed export is caught before a single API call is spent on it — the red chip shows a missing required column, and the error text names the column and what to do about it. A live build of this screen is linked in section 12.

Architecture

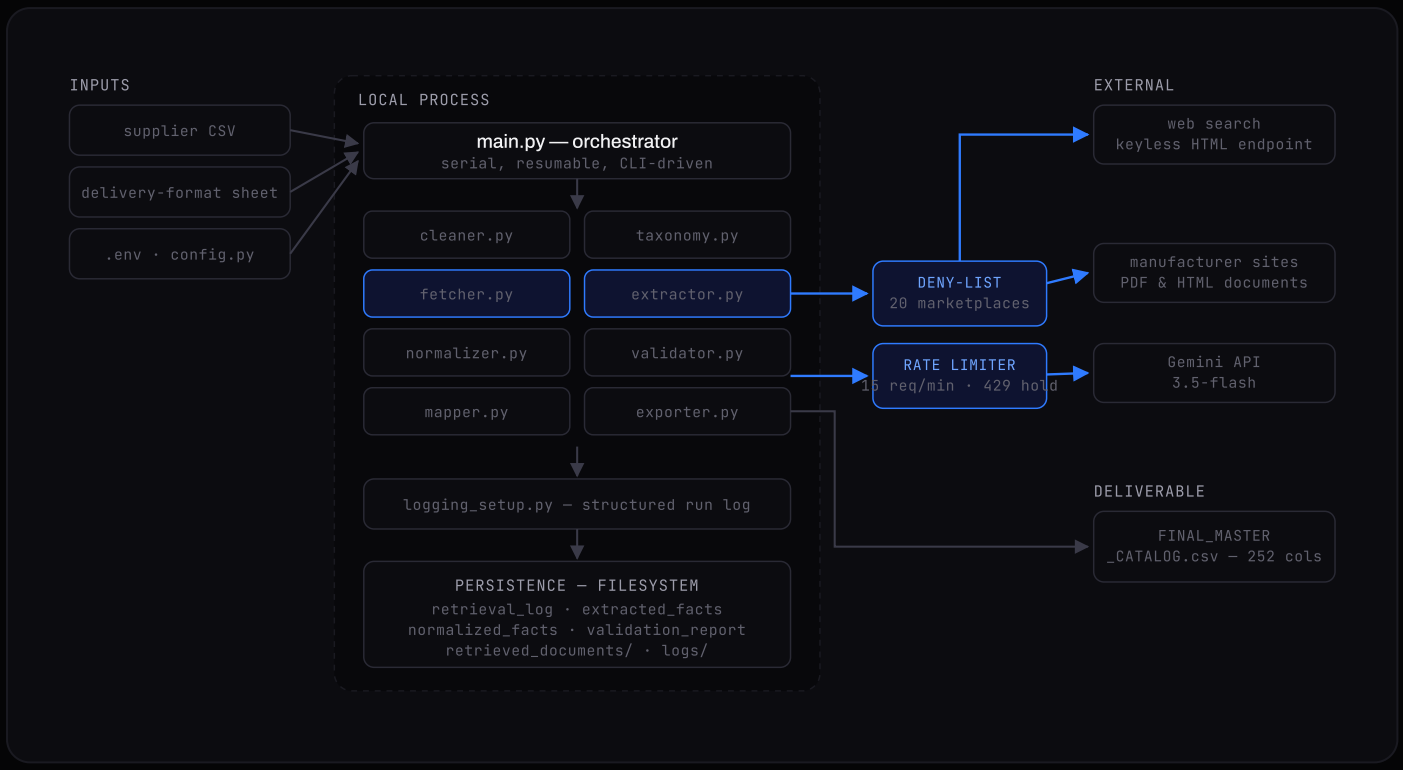

Figure 3. The claim this diagram makes: the trust boundary has exactly two crossings, and both are guarded. Retrieval leaves through a marketplace deny-list; extraction leaves through a rate limiter that also absorbs quota errors. Five of the seven stages never touch the network, which is why the pipeline is deterministic and testable where it matters.

11 — TECHNOLOGIES USED

Stack

Layer	Technology	Why it is there
Language	Python 3.10+	Data-processing ecosystem and the reference SDKs for the model provider
Data handling	pandas	Catalog-scale frames, grouping facts by product, CSV round-tripping
AI extraction	google-genai · Gemini 3.5-flash	Structured JSON extraction at low latency; free tier makes the pilot cost nothing
Document retrieval	requests	Streamed downloads with timeouts and status handling
HTML parsing	beautifulsoup4	Search-result parsing and readable text extraction with nav/script stripped
PDF parsing	pypdf	Page-level text extraction from manufacturer datasheets
Validation models	pydantic	Typed fact schema at the extraction boundary
Configuration	python-dotenv	API credentials kept out of source control
Observability	logging (stdlib) · tqdm	Per-run timestamped log files plus live progress on long runs
Interface	HTML · CSS · vanilla JS · Canvas	No framework, no build step; CSV parsed client-side, ambient field drawn on Canvas
Storage	CSV filesystem	Every intermediate is inspectable in a spreadsheet — deliberate for a pilot; Postgres is the scale path
Testing	Python, dependency-free runner	23 regression tests runnable without installing a test framework

What the working system produces

A. Full pipeline run, with document retrieval enabled

```
🚀 Stage 1 & 2: Cleaning & Classification...
Category distribution: {'Abrasives': 2}
✅ Cleaned & classified 2 products

🚀 Stage 3: Autonomous Document Retrieval...
DCB518ASTS06G: retrieved https://diablotools.com/products/DCB518ASTS06G
DCB518ASTS06G: failed https://americatools.com/... (403 Client Error: Forbidden)
✅ Retrieved 1/2 documents → data/intermediate/retrieval_log.csv

🚀 Stage 4: AI Fact Extraction...
✅ Extracted 12 facts from 2 products in 0.3 min
   (2 API calls, 0 rate-limit pauses, 0 failures)

🚀 Stage 5: Deterministic Normalization...
✅ Normalized 12 facts {'SUCCESS': 12}

🚀 Stage 6: Validation & Quality Scoring...
✅ ● 2 AUTO_APPROVED  |  ● 0 HUMAN_REVIEW  |  ● 0 FAILED

🚀 Stage 7: Final Master Export & Schema Mapping...
Loaded delivery schema: 252 columns, 50 attribute slots
Master catalog assembled: 2 rows x 252 columns
🎉 Pipeline complete → data/output/FINAL_MASTER_CATALOG.csv
```

B. Stage 3 finding a real manufacturer source

```
QUERIES
  "DCB518ASTS06G" "Freud Inc" datasheet pdf
  "DCB518ASTS06G" "Diablo" manual pdf
  "DCB518ASTS06G" datasheet

TOP CANDIDATES (after deny-list and authority ranking)
  https://diablotools.com/products/DCB518ASTS06G
  https://countrylinetools.com/products/diablo-dcb518asts06g-...

rejected: amazon.com, grainger.com, zoro.com

DOC TEXT — 6,011 characters extracted
```

C. Normalization measured before and after

On the 25-fact sample run, the flag distribution moved from 15 SUCCESS / 10 UNMAPPED_ENUM to 25 SUCCESS / 0 UNMAPPED_ENUM after the vocabulary was

extended with the abrasive terms the dataset actually contains and imperial fractions were preserved rather than rejected.

D. Test suite

```
$ python tests/test_cleaner.py && python tests/test_pipeline.py
23 passed — routing boundaries, conflict penalty, fail-closed
behaviour, unit and material mapping, duplicate-attribute export,
rate-limiter spacing, retry-delay parsing
```

E. The interface

A live build of the upload screen is published at the uniLog front end. Drop the sample dataset onto it during the demo — it parses all 1,000 rows in the browser, validates the header against the input schema and shows the 6 → 252 projection. Screenshot it for the slide, or run it live.

> **Before the presentation:** run python main.py once over the full catalog and screenshot the final routing summary. Every figure above is real, but they come from runs of two to five products. A full-catalog screenshot is the single strongest slide you can add, and it takes about an hour of unattended runtime.

---

13 — FUTURE DEVELOPMENT

Roadmap

NEAR TERM

Commercial search API

Swap the keyless endpoint for Brave, Serper or Bing. Contained to one function; removes the pipeline's only informal dependency.

NEAR TERM

Headless rendering

Some manufacturer pages build their spec tables with JavaScript and return only a title to a plain fetch. A headless browser recovers them.

NEAR TERM

Measured classification accuracy

Hand-label a few hundred products to produce a defensible accuracy figure instead of an asserted one.

SCALE

Concurrent extraction

Extraction is serial only because of the free-tier quota. On a paid tier a worker pool cuts wall-clock time proportionally with no architectural change.

SCALE

Database backend

Postgres or DuckDB replaces the CSV intermediates past roughly 100,000 products, enabling incremental queries and partial re-runs.

SCALE

Embedding-based classification

Replace keyword rules with vector similarity against category exemplars, so new categories are added by example rather than by code.

QUALITY

Duplicate and variant detection

Identify products that are the same item under different supplier part numbers, and group true variants into families.

QUALITY

Review console

A queue interface for the amber tier: the flagged product, its evidence, the conflicting values, and one click to accept or correct.

COVERAGE

Image and asset pipeline

The delivery format has columns for product images, alternate images, spec sheets and manuals. Retrieval already finds the documents; the next step is naming and storing them to the client's convention.

---

14 — ENTERPRISE SCALABILITY

What makes it scalable

The property that matters most: **processing is stateless per product.** No stage needs to see product N−1 to handle product N. That single design choice is what makes every scaling answer below straightforward.

Large product catalogs

- **Embarrassingly parallel by construction.** Because products do not interact, the work shards trivially across workers or machines. The pipeline runs serially today for one reason only — a 15 request/minute free-tier quota — which is a commercial limit, not an architectural one.
- **Runs are resumable.** Each stage writes its output before the next begins, so a failure at product 800 does not discard the first 799. Extraction — the expensive stage — can be re-entered from extracted_facts.csv without re-spending a single API call.
- **Cost scales linearly and predictably.** One API call per product, so a catalog's cost is a multiplication, not an estimate. The four local stages are pure CPU and effectively free.
- **The storage path is known.** CSV is a deliberate pilot choice because every intermediate is inspectable in a spreadsheet. Past roughly 100,000 products the same code writes to Parquet or Postgres — the frames and the interfaces between stages do not change.

New manufacturers

- **Nothing is hardcoded per manufacturer.** There is no per-vendor URL table to maintain. Discovery composes its queries from the part number and the resolved manufacturer, and ranking simply prefers whichever domain matches the brand token. Onboarding a manufacturer that has never been seen requires zero code and zero configuration.
- **The source policy is a deny-list, not an allow-list.** This is the deliberate choice that makes new manufacturers work by default: any legitimate domain is permitted unless it is a known marketplace. An allow-list would require curation for every new vendor and would silently drop unknown ones.
- **Brand resolution is evidence-driven.** Brands are recovered from the product text rather than from a maintained brand dictionary, so an unfamiliar brand resolves on its first encounter.

> **The honest caveat.** Category classification is the part that does need attention as the catalog widens — it is keyword-driven, so a genuinely new product category needs rules added. It is an isolated lookup table rather than logic threaded through the codebase, and the embedding-based replacement in the roadmap removes the limitation. Say this before a judge finds it.

Different document formats

- **Extraction is deliberately two steps: format → text, then text → facts.** Only the first step knows about file formats. Adding one means writing a reader that returns a string; the prompt, the normalizer, the validator and the exporter are untouched.
- **Shipping today:** PDF via pypdf and HTML via BeautifulSoup, with content-type detection choosing the reader automatically.
- **The same seam accommodates** DOCX, XLSX specification tables, and OCR for scanned datasheets — all common in industrial supply, all just another reader.
- **Failures are contained.** An unreadable document is logged with its reason and the product falls back to supplier text rather than crashing the run, so one corrupt PDF in ten thousand costs one product's enrichment, not the batch.

Continuous product updates

- **Re-runs can be selective, and the score decides which.** Products already sitting green with high confidence need no reprocessing. Re-run the amber and red tiers, and anything whose source document has changed. The quality score is already the prioritisation signal — no separate scheduling logic is needed.
- **Change is detectable.** Retrieval writes a timestamped log with the source URL and local file path for every document, so a later run can compare against what it fetched before and re-extract only where the source actually moved.
- **The delivery format already carries update semantics.** Its Action_Code column distinguishes an inserted record from an updated one, so downstream systems can consume deltas rather than full reloads.
- **New supplier columns do not require a release.** Because the output schema is read from the client's sheet each run, a revised delivery format is picked up automatically on the next

run.

Stateless per product, deny-list not allow-list, and a format seam one function wide. Those three choices are the whole scalability answer.